Zoty GraphQL

Zakladni informace

Jak je mozne graphql volat je popsano v clanku "4 Simple Ways to Call a GraphQL API".
Jak graphql funguje je mozne si precist napriklad v tomto navodu.

V ramci https://zoty.cloud/graphql/?access_token= je mozne si vyzkouset praci s graphql.

Bearer token

Nejprve je nutne ziskat token pro overeni uzivatele.

```
mutation {
  loginMutation(input: {
    email: "abcd@xyz",
    password: "xxxxxx",
    accountId: "xxxxxx",
  })
}
```

Vraceny token ABCD zkopirujeme do URL a s nim muzeme pristupovat k dalsim query graphql pres https://zoty.cloud/graphql/?access_token=ABCD.

Napriklad seznam typu entit v Zoty.

```
query {
  entityTypeListQuery(input: {}) {
    name
  }
}
```

Obdobne je mozne si vyzkouset overeni uzivatele pres CURL.

```
curl \
  --request POST \
  --header 'Content-Type: application/json' \
  --data '{"query": "mutation login($input: LoginInputType!)
{loginMutation(input: $input)}", "variables": {"input": {"email": "xxxxxx",
"password": "xxxxxx", "accountId": "xxxxxx"}}}' \
  https://zoty.cloud/graphql
```

A seznam vsech typu entit v Zoty pres

```
curl \
  -X POST \
  -H "Content-Type: application/json" -H "authorization: Bearer ABCD" \
  -d '{"query": "{entityTypeListQuery {id, name}}"}' \
  https://zoty.cloud/graphql
```

kde ABCD je ziskany token z predesleho requestu.

Rozhrani Zoty Graphql (https://zoty.cloud/graphql?access_token=ABCD) prinasi dokumentaci vsech schemat, ktere je mozne v Zoty vyuzit pro ziskani dat z databaze PostgreSQL. K dispozici jsou Query i Mutation dle opravneni uzivatele. Ziskavame informace, co je mozne pozadovat v ramci response (`id`, `name`,...) a jake jsou potrebne vstupni `input` data.

Puvodni query muzeme rozsirit a pozadat GraphQL o vraceni vice informaci z DB.

```
query {
  entityTypeListQuery(input: {}) {
    id
    name
    methodIdList
    stateList {
      stateId
    }
    ...
  }
}
```

Nektere tabulky/Query (activityListQuery, entityListQuery, externalSourceListQuery, relationListQuery) mohou byt v Zoty velmi rozsahle a proto nepristupujeme ke vsem zaznamum najednou a snazime se je z DB ziskavat az v ramci urciteho kontextu.

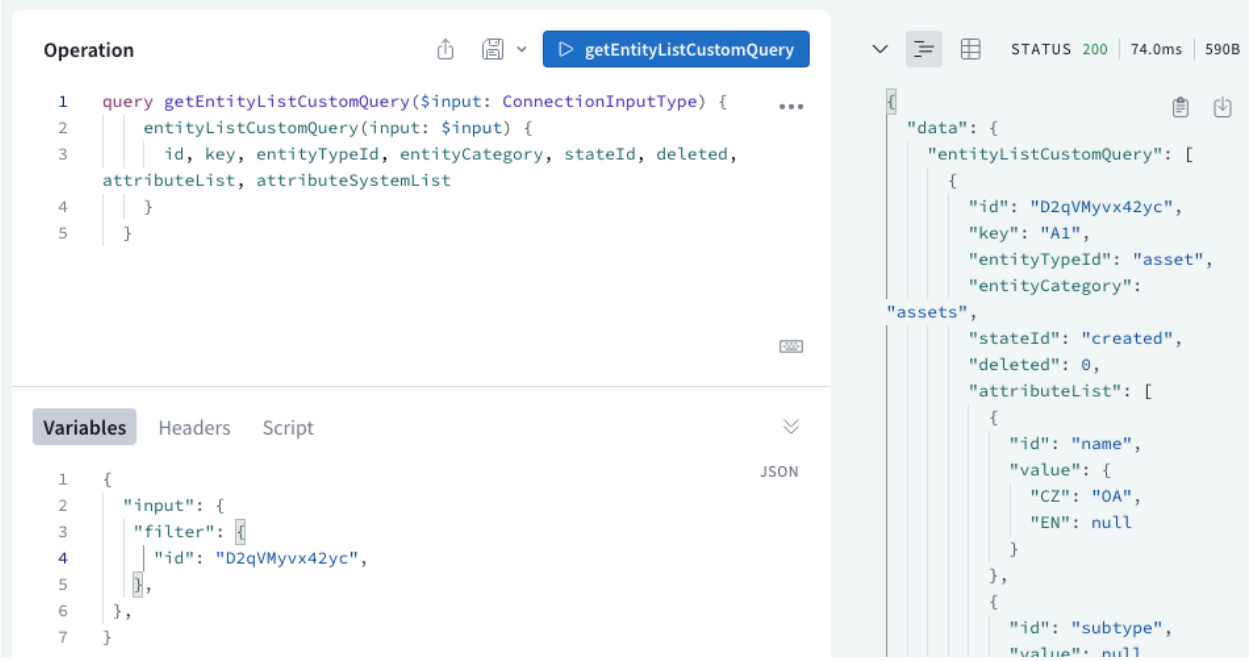
Filtr

K filtrovani dat, ziskani urciteho kontextu, je doporucene vyuzit `input` objekt s klicem `filter`.

```
input: {
   filter: {
     id: idList,
   },
 },
```

Nejprve je ale nutné zkontrolovat, že input query obsahuje `ConnectionInputType`, který mimo jiné slouží k “aktivaci” filtru pres `"variables"`.

```
query EntityListQuery($input: ConnectionInputType) {
  entityListQuery(input: $input) {
    id
    entityTypeId
  }
}
```



Filtrování dle atributu `"entityTypeId"`, který obsahuje každá entita.

```
{
  "input": {
    "filter": {
      "entityTypeId": ["asset", "task"],
    }
  }
}
```

Pokud ve filtru potřebujeme “nebo”, máme k dispozici operátor `"$or"`. Operátor se využívá pro filtrování databáze na základě více kritérií, zpravidla rozdílných atributů. Výsledek bude stejný jako výše.

```
{
  "input": {
    "filter": {
      "$or": [
        { "entityTypeId": "asset" },
        { "entityTypeId": "task" }
      ]
    }
  }
}
```

Operator lze ruzne kombinovat

```
{
  "input": {
    "filter": {
      { "relationTypeId": "relationship" },
      "$or": [
        { "sourceId": "AAAA" },
        { "targetId": "BBBB" }
      ]
    }
  }
}
```

Typy operatoru

`"$exists"` (whereNotNull), `"$gt"` (>), `"$gte"` (>=), `"$in"` (whereIn), `"$like"` (LIKE), `"$lt"` (<), `"$lte"` (<=), `"$ne"` (whereNot), `"$nin"` (whereNotIn), `"$or"` (orWhere), `"$json"` (->, ->>, #>, #>>), `"$raw"`.

```
{
  "input": {
    "filter": {
      "id": {
        "$nin": ["asset"],
      },
    }
  }
}
```

Specifické jsou `"$json"` a `"$raw"`, které jsou spojeni s jménem atributu. Ve stringu pro `"$raw"` je nutné vždy uvést `this`, který je nahrazen jmenem atributu.

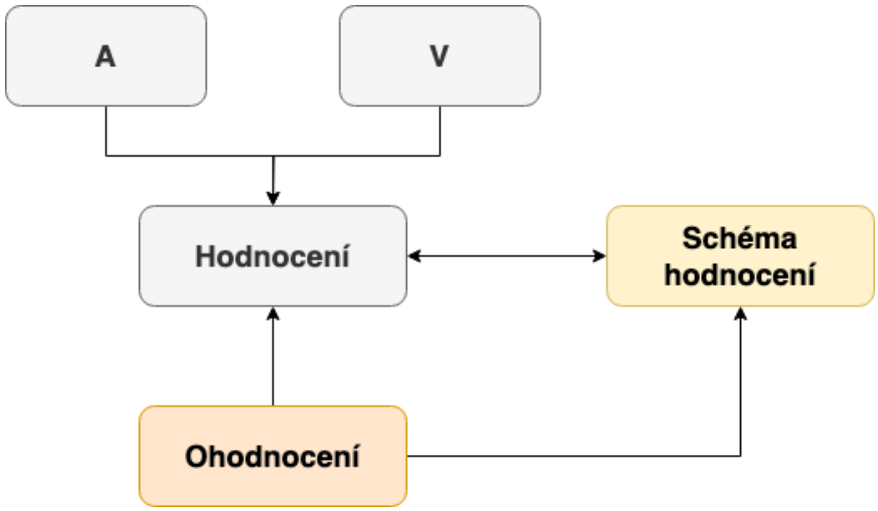
```
{
  "input": {
    "filter": {
      "metadata": {
        "$json": "->>'objectKey' IS NOT NULL",
      },
    }
  }
}
```

`this` bude nahrazeno za `methodIdList`.

```
{
  "input": {
    "filter": {
      "methodIdList": {
        "$raw": "this && array['methodId', 'method2Id']::varchar[]",
      },
    }
  }
}
```

Ohodnoceni

Ohodnoceni je konkretni hodnoceni pro zvolenou casovou periodu (1. 1. 2024 - 31. 1. 2024,...). Ohodnoceni je vytvorene dle konkretniho Schema hodnoceni (Hodnocení rizik dle ISO 31000,...). Schema je vytvoreno pro entity typu Hodnoceni (entityTypeId = ‘evaluation’). Konkretni entita Hodnoceni je propojena s rodicem (entity Aktiva, Rizika, Hrozby, Zranitelnosti).



Napriklad v query `evaluationListQuery` pro jednotliva ohodnoceni, lze ziskat jen ta, ktera se tykaji konkretni jedne entity Hodnoceni. Hledame ID entity `AAAA` a input muze byt

```
input: {
   entityId: "AAAA",
 },
```

Nebo vyuzijeme `filter` kde v evaluationEntityIdList (`evaluationEntityIdList = [AAAA, bbbb, cccc];`) posleme pole ID entit. Ziskavame zaznamy ohodnoceni pro vsechny tyto entity (pokud existuji).

```
input: {
   filter: JSON.stringify({
     entityId: {
       $in: evaluationEntityIdList,
     },
   }),
 },
```

Nicmene `input` neni povinny a kdyz se query zavola bez konkretni entityId / schemaId / filter, tak nam GraphQl vrati vsechna ohodnoceni, ktera v DB existuji.

```
query {
  evaluationListQuery {
    id
  }
}
```

Ohodnoceni pres rodice

V Zoty muzeme ziskat ohodnoceni skutecnosti, nebo cile primo pres rodice (Rizika, Aktiva, Hrozby, Zranitelnosti) entity Hodnoceni. Vyuzit je mozne query `evaluationListByParentsQuery`, respektive jeji filter.

```
query ($input: EvaluationListByParentsInputType) {
  evaluationListByParentsQuery(input: $input) {
    parentEntityIdList
    score {
      levelType
    }
    type
  }
}
```

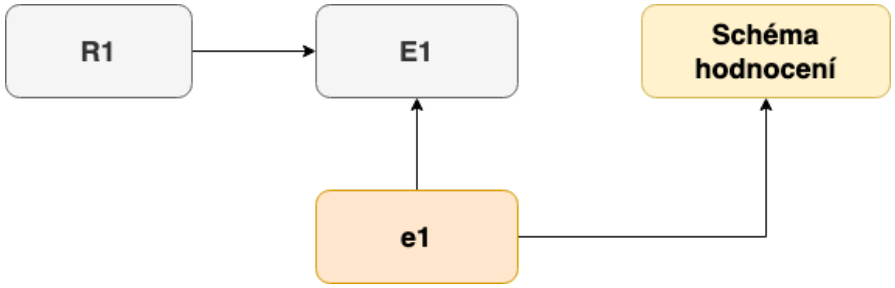
```
{
  "input": {
    "filter": {
      "parentEntityIdList": ["RIZIKO|AKTIVUM|..."],
      "type": ["current"],
    }
  }
}
```

Entity typu “Hodnoceni”

Jednotliva "ohodnoceni" `evaluationListQuery` jsou propojena pouze s entitami typu "Hodnoceni" (`entityTypeId` = `evaluation`). Ty pak jsou propojeny vazbou parent s rodicem (entity s `entityCategory` aktiva - `assets` / hrozby - `threats` / zranitelnosti - `vulnerabilities` / rizika - `risks`).

V evaluation nalezneme v `entityId` vzdy entitu typu Hodnoceni, ktera byla vytvorena pro konkretni spojeni rodicu, atribut `parentEntityIdList`, zde nalezneme ID entit Aktiv, Zranitelnosti, Hrozeb, Rizik. Entita Hodnoceni muze byt vytvorena pro konkretni Aktivum, nebo dohromady se Zranitelnosti, nebo pro Riziko, ktere ma za rodice Aktivum, Zranitelnost,...

Entita typu hodnoceni (E1), vytvorena pro konkretni riziko (R1), ktere hodnotime pres Ohodnoceni (e1) je propojeno jako R1 -> E1 -> e1. S tim, ze riziko R1 muze mit take sve rodice.




```
query {
  evaluationEntityListQuery(input: {
    fieldList: ["attribute.agent"],
    filter: {
      id: "DJM3183g1YYn",
    }
  }) {
    id
    attributeList
    entityTypeId
    relationList {
      relationTypeId
      sourceId
      targetId
    }
    evaluationList {
      schemaId
```

```
      entityId
      parentEntityIdList
      score {
        levelType
        value
        valueYX
      }
      type
    }
  }
}
```

Vysledkem query vyse je


```
{
  "data": {
    "evaluationEntityListQuery": [
      {
        "id": "DJM3183g1YYn",
        "attributeList": [{
            "id": "agent",
            "value": null
          }, {
            "id": "subtype",
            "value": "evaluationA"
        }],
        "entityTypeId": "evaluation",
        "relationList": [{
          "relationTypeId": "parent",
          "sourceId": "R1",
          "targetId": "E1"
        }],
        "evaluationList": [{
          "schemaId": "5zLzncuuqwki",
          "entityId": "E1",
          "parentEntityIdList": ["R1"],
          "score": {
            "levelType": "negativeModerate",
            "value": 0,
            "valueYX": 2
          },
          "type": "current"
```

```
        }]
      },
```

Vidime, ze ohodnoceni (e1) v `evaluationList` je “propojeno” na entitu Hodnoceni `"E1"` a nese informaci pro ktere rodice (R1) entity Hodnoceni bylo vytvoreno `"parentEntityIdList": ["R1"]` . Entita Hodnoceni ma propojeni v `relationList` typu `"parent"` prave na ID `"R1"`.

Entity

Kazda entita ma podle sveho typu prirazeny ruzne atributy. Mnoho atributu je stejnych napric ruznymi typy, jako jeji nazev, popis a jine. Nektere jsou specificke dle konkretniho typu entity, nebo prirazene metode. Prirazena metoda tak muze entity dale rozsirovat o dalsi atributy.

Seznam typu entit

Pokud se podivame na query pro seznam vsech typu entit `entityTypeListQuery`, tak v `attributeList` vidime vsechny atributy, ktere lze u entity konkretniho typu vyplnit, ktere jsou k entity typu prirazeny.

```
query {
  entityTypeListQuery {
    id
    name
    isSystem
    attributeList {
      id
      attribute {
        id
        name
        type
      }
    }
    entityCategory
    methodIdList
  }
}
```

Kdyz query `entityTypeListQuery` rozsirime o `type`, ziskavame informaci jakeho typu atribut je. Muze byt boolean, datetime, float, json, relation, string a text.

Atributy v seznamu entit

Diky `entityTypeListQuery` vidime napriklad u rizika, ze muze mit atribut validFrom.
Ten je mozne pridat u `entityListQuery` do pole `fieldList` s prefixem attribute `"attribute.validFrom"`. Vsechny zakladni atributy, ktere jsou ulozeny primo u entity jsou volany pres `"attribute.*"`.

```
query {
  entityListQuery(input: {
    fieldList: ["attribute.validFrom", "attribute.name"],
    filter: "{\"entityCategory\": \"assets\", \"entityTypeId\": {\"$nin\":
[\"folder\", \"rootFolder\"]}}"
  }) {
    id
    attributeList
    attributeSystemList
  }
}
```

Filter je zde zadan ve formatu JSON aby bylo mozne definovat primo v Query specificky klic `$nin`. Query vyse vyhleda vsechny entity, ktere maji kategorii Aktiva a nejsou typu Slozka.

Pokud ma entita u tohoto atributu hodnotu vyplnenou, bude v ni zobrazeno datum, jinak bude `"value": null`. Pokud entita, respektive jeji typ, tento atribut vubec nema prirazen, bude i u ni tento atribut zobrazen (protoze je vyse zavolan a nelze jej v tomto pripade vyfiltrovat podle entity typu, jak to automaticky dela `entityListQuery`) a bude take u nej hodnota `null`.


```
{
  "data": {
    "entityListQuery": [
      {
        "id": "AKTIVUM",
        "attributeList": [
          {
            "id": "validFrom",
            "value": null
          },
          {
            "id": "name",
            "value": {
              "CZ": "Informace #1",
```

```
            "EN": null
          }
        },
```

Obdobne je mozne se podivat na deadline u ukolu. Ten ma v zakladnim nastaveni take pouze entita typu task. Proto jine entity budou mit tento atribut zobrazen, ale nebude vyplnen.

Relacni atributy

Query `entityListQuery` muze ve `fieldList` obsahovat mimo “klasicke” atributy `"attribute.*"` take relacni atributy. Ty nejsou primo ulozeny v tabulce entity.entity ale v jinych relacnich tabulkach.
Seznam relacnich atributu nalezneme dle vyplneneho typu v tabulce entity.attribute.type s hodnotou “relation”. Tyto atributy ziskavame pres `"attributeRelation.*"`.

```
query {
  entityListQuery(input: {
    fieldList: ["attributeRelation.guarantor"],
  }) {
    id
    attributeList
  }
}
```

```
{
  "data": {
    "entityListQuery": [
      {
        "attributeList": [
          {
            "id": "guarantor",
            "value": "CCDFNukeGRxa",
            "valueType": "user",
            "valueAdditional": {}
          }
        ]
      },
```

Specificke atributy

Specialni atributy pro `fieldList` jsou tzv. `"custom.*"`, ktere jsou definovany pro specificke ucely a vysledne hodnoty nalezneme v `attributeSystemList`.

Vytvoreny jsou `"custom.timelineMap[stateId,...]"`, kde `stateId` lze zamenit, nebo doplnit o dalsi atributy. Funkce pracuje s Aktivitami a vraci u entit vsechny zmeny v `stateId`.

Data z tabulky entity.attribute_period lze ziskat pres `"custom.attributePeriodList"`. Jedna se o periodicke atributy, ktere reprezentuji hodnotu, ktera se v definovanem case opakuje. Napriklad naklady entity jako mzda, ktera se vyplaci kazdy mesic. Z hodnot muzeme pocitat rocni naklady a podobne.

Naklady

Konkretne naklady je ale mozne ziskat pres `attributeSystemList`, kde jsou automaticky vypocteny rocni, nebo aktualni naklady, spolecne s CAPEX a OPEX. Nutne je vyuzit query `entityListQuery`.

Tagy a Organizace

K entite muzeme pridat Stitky `tagList` a Organizace `organizationList`. Oboje jsou specifickym pripadem Skupin, do ktere je mozne entity rozradit.

```
query {
  entityListQuery(input: {
    filter: "{\"entityCategory\": \"tasks\"}"
  }) {
    id
    tagList {
      id
      name
    }
    organizationList {
      id
      name
    }
  }
}
```

Aktivity

Aktivity je nejrozsahlejsi tabulka v uctu zakaznika. Kazda zmena, ktera je u entity provedena je zde zaznamenana.

Pro pristup k datum doporucujeme `activityListCustomQuery`.
Je osetrena, at uzivateli neni zobrazeno vse v teto tabulce, protoze response by trvala velmi dlouho a byla narocna na vykon.

Take doporucujeme vyuzit filter, ktery je u tohoto modelu mozny u id, parentType, parentId, authorId, activityType, createdAt.

```
query {
  activityListCustomQuery(
    input: {
      filter: {
        parentId: ["AAAA", "BBBB"]
        activityType: "update"
      }
    }
  ) {
    id
    parentId
    activityType
    changes
  }
}
```

Filtrujeme jen zaznamy, ktere odpovidaji ID entitam v poli `parentId`.
K tomu chceme ale jen ty zaznamy, ktere maji `activityType` specialne jen `"update"`, kdy vime, ze aktivita obsahuje upravu danych entit.

V response dostaneme:

```
{
  "data": {
    "activityListCustomQuery": [
      {
        "id": "6666t2L6r3cr",
        "parentId": "AAAA",
        "activityType": "update",
```

```
        "changes": {
          "stateId": {
            "value": "finished",
            "valueOld": "inProgress"
          }
        }
      },
...
```

Zde vidime vsechny zmeny `"changes"`, ktere Aktivita obsahuje pro konkretni entitu. Zmena byla provedena u atributu `"stateId"`, kde puvodni hodnota `"inProgress"` byla nahrazena novou hodnotou `"finished"`.

Pokud pridame `final`, mame kompletni objekt entity po zmene a v `original` puvodni objekt entity pred zmenou.

Filtr s operatorem $json

Pro detailnejsi filtrovani Aktivit je mozne vyuzit rozsireny JSON filter https://www.postgresql.org/docs/9.5/functions-json.html.
Napriklad pro 2 entity (AAAA a BBBB, pripadne vice) je mozne si zavolat vsechny Aktivity, ktere obsahuji zmeny stavu.

```
{
  "input": {
    "filter": {
      "changes": {
        "$json": "->>'stateId' IS NOT NULL"
      },
      parentId: ["AAAA", "BBBB"]
    }
  }
}
```

Vrati se vsechny zaznamy, kde parentId obsahuje AAAA nebo BBBB a aktivita ma v atributu `changes` zmenu v `stateId`.
V tomto objektu mohou byt i dalsi zmeny, pokud byly upraveny ve stejny okamzik. Nas ale v tomto kontextu zajima pouze `value` a `valueOld` u `stateId`.

```
{
  "data": {
    "activityListQuery": [
```

```
      {
        "id": "71KW7UnMzs9Y",
        "parentId": "AAAA",
        "activityType": "update",
        "changes": {
          "stateId": {
            "value": "inProgress",
            "valueOld": "taken"
          }
        },
        "createdAt": "2019-06-06T11:55:41.665Z"
      },
...
```